Lab03. Containers.

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning goals

1. Running simple containers.
2. Create container image.
3. Run container from the image.
4. Orchestrate multiple containers.

2 Prerequisites

Before the exercise, we will make our Cloud9 virtual machine slightly bigger (change site from t2.)

1. Close all Cloud9 tabs, if you have opened any.
2. In AWS Console open EC2 service.
3. Find the EC2 instance related to your Cloud9 environment.
4. Select this EC2 instance and change its state to Stopped. If you cannot see the instance, disable the filter that shows only running instances.
5. Change the size of the EC2 instance:
    1. Select: Actions > Instance settings > Change instance type
    2. Set "New instance type" to `t2.small` (or `t3.small` if `t2.small` is not available).
    3. Click "Change" button.
6. Open security group assigned to Cloud9 EC2 instance (select EC2 instance, then open Security tab at the bottom).
7. Edit inbounds rules. Add two inbound rules:
    1. Custom TCP - port 5000 - source 0.0.0.0/0
    2. Custom TCP - port 3000 - source 0.0.0.0/0

   Save security group settings.
8. Start your Cloud9 IDE.
9. In Cloud9 open the second Terminal (select "Window > New Terminal" from menu or press Alt-T).
10. If present, remove `cloud9-demo/` folder from the `environment/` folder.

Explanation: Cloud9, by default, forwards local ports 8080-8082 to the Internet, to review the results of locally working application. To not break this functionality, we will use ports 3000 and 5000 for our containers instead.

3 Exercises

3.1 Running containers

In the first part, you will start containers with different Linux distributions: Debian, Fedora, Alpine. For each distribution you install `stress-ng` application. You will also save one of the modified containers as a new container image, to create new containers with changed content.

1. Check the version and configuration of Docker using command: `docker version`
2. Start the container using Debian image: `docker run -it debian /bin/bash` . Pay attention, how long does container start.
3. Execute the following commands and observe the results:

```
# hostname
# cat /etc/os-release
# apt update
# apt install stress-ng
# echo "Hello" > myfile.txt
# cat myfile.txt
```

4. Open a new terminal (menu "Window > New Terminal" or press Alt-T) and execute command: `docker ps` . Compare the `ContainerID` and the result of the `hostname`.

   What is the name of the container?

5. In the first terminal (Debian) type `exit`
6. Execute: `docker ps`. Can you see your container?
7. Execute: `docker ps -a`. Can you see your container now?
8. Execute: `docker start -ai <CONT-ID>` (replace `<CONT-ID>` with ID of your container). When inside container execute: `cat myfile.txt`
9. Type `exit` to exit the container.
10. Execute: `docker commit <CONT-ID> myimage` to save modified container into new *container image* with new name.
11. Execute: `docker run -it myimage /bin/bash` , then inside the container: `cat myfile.txt` .

    Does file exist? What is the content of the file? What is ID and the name of the second container? Are they the same as previously?

12. Execute: `docker run -it fedora /bin/bash` . Execute all commands from the step 3.

    Were you successful?

13. Try the following commands: `yum update` and `yum install stress-ng`. Exit from the container.
14. Execute: `docker run -it alpine /bin/sh` . Execute all commands from step 3.

    Were you successful?

15. Try `apk update` then `apk add stress-ng`.

    Exit from the container.

16. List all container images stored in your machine: `docker image ls` . Compare sizes of container images.

3.2 Run the container with options

In this exercise, you will start container using Bitnami Apache image, to start Apache web server (httpd) in container, publish its ports to your host system and share files between container and your computer.

1. In the terminal, change folder to `/home/ec2-user/environment` .
2. Create new folder: `mkdir my-site`
3. Inside the `my-site` folder, create `index.html` file, and put the following content inside:

```
<h1><YOUR-INITIALS></h1>
<p>This is my Bitnami Apache serwer</p>
```

4. Start a new container with Apache web server:

```
$ docker run --name apache -v /home/ec2-user/environment/my-site:/var/www/html -p 8080:80
↪  ubuntu/apache2
```

   Option `-v` mount folder from your system into containers folder.

   Option `-p` publishes (forwards) container's port (80) to EC2 instance port (8080).

   Option `--name` gives your container a constant name, instead of random name assigned by Docker.

5. In Cloud9 click in the middle of the top bar "Preview", and then select "Preview running application". You should see your application running in Cloud9 subwindow.
6. Using Cloud9 editor change the contents of `index.html` file, save the file, and refresh the browser. Can you see the changes?
7. Open the second Bash terminal and remove the container from the memory: `docker rm apache -f`
8. Run the container in the background:

```
$ docker run --name apache -d -v /home/ec2-user/environment/my-site:/var/www/html -p
↪  8080:80 ubuntu/apache2
```

   When you use `-d` option, container runs in the background, so you can still use the terminal.

9. Remove all containers from memory.

3.3 Build your own container image

1. Clone the repository `cloud9-demo` into `environment/` folder:

```
$ git clone https://github.com/pwr-cloudprogramming/cloud9-demo.git
```

2. In the left pane expand folder `cloud9-demo/backend/`.

3. Open file `Dockerfile`, and study its content.
4. In terminal change directory to `backend/`
5. Build the container image (pay attention to the dot!):

```
$ docker build -t mybackend:v1 -t mybackend:latest .
```

6. Run the container using your image:

```
$ docker run --name backend -p 5000:5000 mybackend
```

   Open the backend web page in your browser.

7. Stop your container using command `Ctrl-C`. Remove the container from memory: `docker rm backend`
8. Open `app.py` file, and insert your last name (surname) into `<h1>` tag.
9. Build a modified container image using command:

```
$ docker build -t mybackend:v2 -t mybackend:latest .
```

10. Run a container again:

```
$ docker run --name backend -p 5000:5000 mybackend
```

    List container images. Which image was used? Were you able to see your last name?

11. Stop the last container, and remove it from the memory: `docker stop backend && docker rm backend`.
12. Start the container using command:

```
$ docker run --name backend -p 5000:5000 mybackend:v1
```

    What version is visible in the browser now? Can you see your last name?

13. Open the `frontend/` folder. Compare the content of both `Dockerfile` files. Especially, pay attention to the first line `FROM`.

    In `index.html` file, replace `<CLOUD9-PUBLIC-IP>` with the public IP address of EC2 instance. Replace `YOUR-INITIALS` with your initials. Then build frontend container image and run container:

```
$ cd ../frontend/
$ docker build -t myfrontend:v1 -t myfrontend:latest .
$ docker run --name frontend -p 3000:3000 myfrontend
```

14. Verify, if frontend displays information retrieved from backend (current time).

    Demonstrate web page, SSH connection and EC2 instance to your teacher.

3.4 Orchestrate containers using docker-compose

1. Remove all working and stopped containers from your system, and check if all were removed:

```
$ docker rm -f $(docker ps -a -q)
$ docker ps -a
```

2. In Bash terminal, change directory to `cloud9-demo/` folder.
3. Install `compose` plugin in Cloud9:

```
$ sudo mkdir -p /usr/local/lib/docker/cli-plugins

$ sudo curl -sL https://github.com/docker/compose/releases/latest/download/docker-compose-
↪  linux-$(uname -m) -o /usr/local/lib/docker/cli-plugins/docker-compose

$ sudo chown root:root /usr/local/lib/docker/cli-plugins/docker-compose

$ sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

   Warning: The second `curl` command contains very long URL address. Copy it into text editor and join it into one long line. Then paste in into Bash terminal.

4. Review the content of the `compose.yaml` file.
5. Run command `docker compose up -d` to start both backend and frontend in the background. Option `-d` states for *daemon* – Unix application running in the background (service).
6. List all running containers.
7. Open frontend and backend in the browser.
8. Stop all modules with command `docker compose down`
9. When you changed the code, and you want to update container images, you run command: `docker compose up -d --build`